FACULDADE DE TECNOLOGIA DO IPIRANGA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (NOTURNO)

ESTRUTURA DE DADOS

Atividade N1-2

Lucas Bastos Palmaccio

São Paulo, SP
23 de março de 2026

Lucas Palmaccio

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
```

main

A	pointer to No ?
B	pointer to No ?

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

Lucas Palmaccio

Este código demonstra a alocação dinâmica de dois nós (A e B), o encadeamento entre eles e o efeito colateral (falha de segurança) ocorrido após a liberação da memória de um dos nós.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

main

A	pointer to No ?
B	pointer to No ?

criar_no

valor	int ?
novo	pointer to No ?

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

Lucas Palmacio

A função malloc reserva espaço no Heap.

- A verificação if (novo != NULL) garante que a memória foi alocada com sucesso.
- A atribuição novo->proximo = NULL; evita que o ponteiro carregue "lixo" de memória.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

main

A	pointer to No ?
B	pointer to No ?

criar_no

valor	int 10
novo	pointer to No ?

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

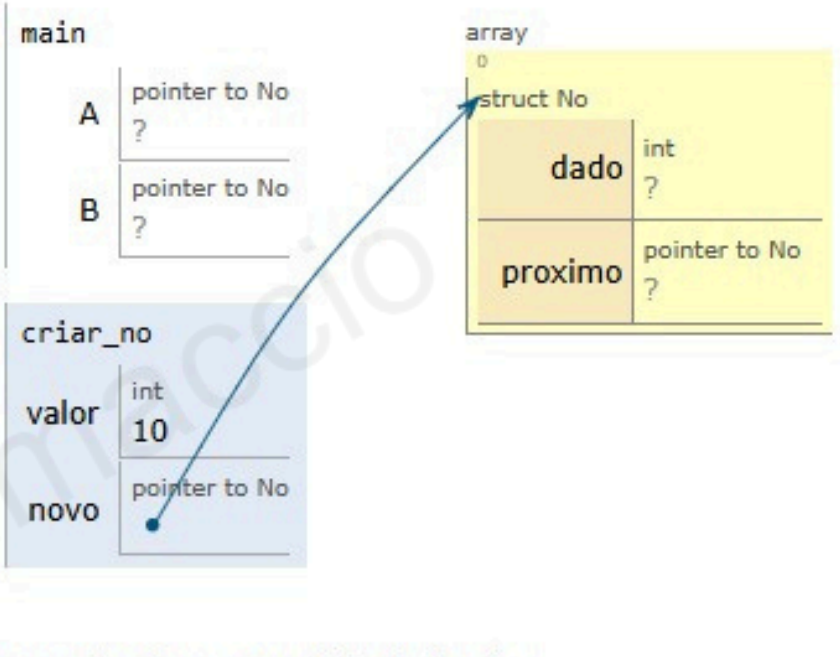
Lucas Palmaccio

A função malloc reserva espaço no Heap.

- A verificação if (novo != NULL) garante que a memória foi alocada com sucesso.
- A atribuição novo->proximo = NULL; evita que o ponteiro carregue "lixo" de memória.

known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



Note: ? refers to an uninitialized value

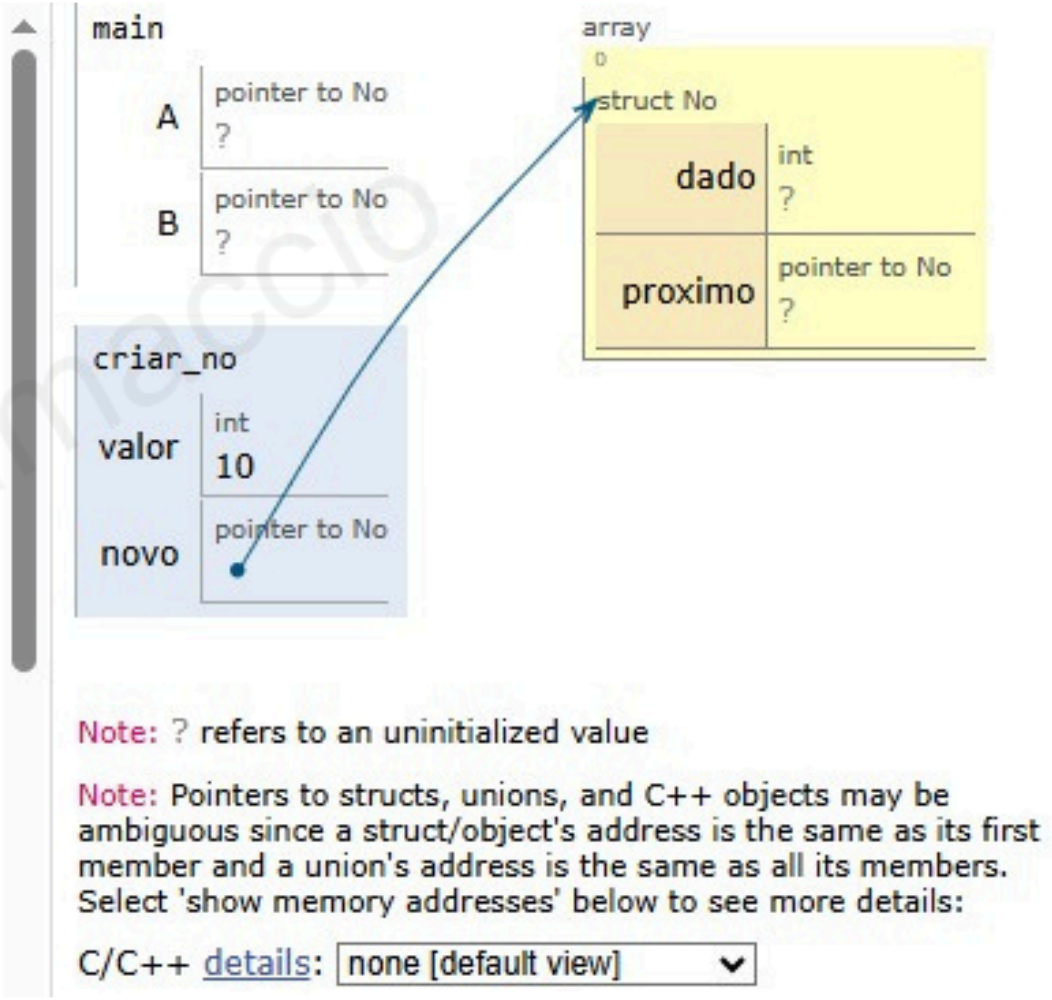
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

Lucas Palmeccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

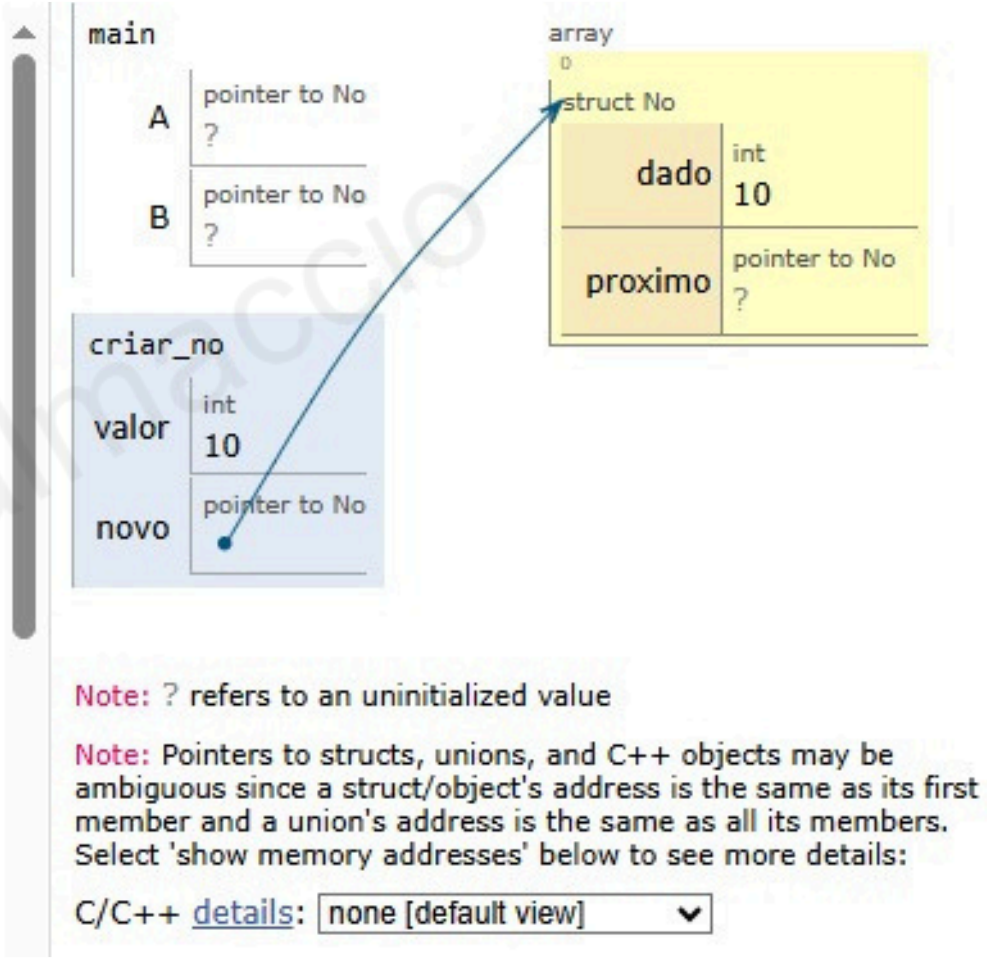
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



Lucas Palmaccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

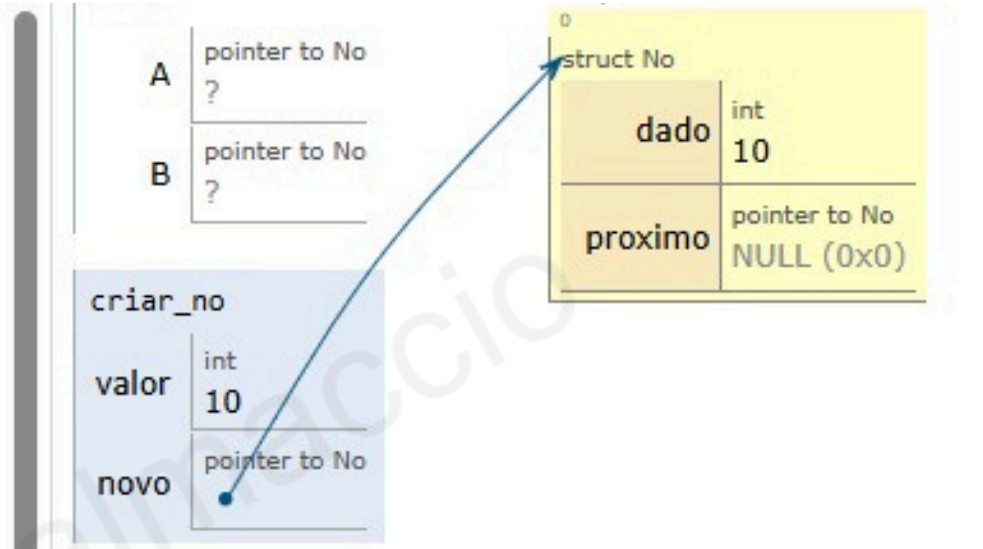
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



Lucas Palmaccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



Note: ? refers to an uninitialized value

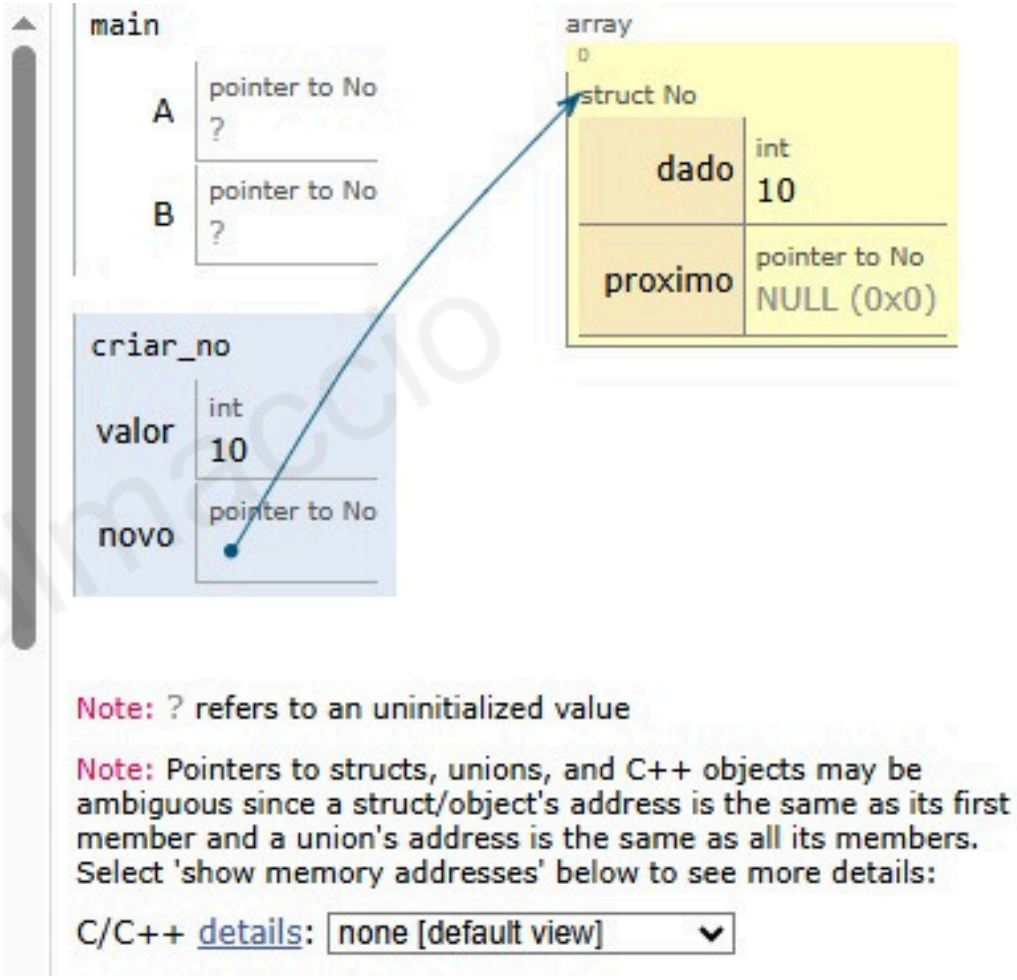
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

Lucas Palmaccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```

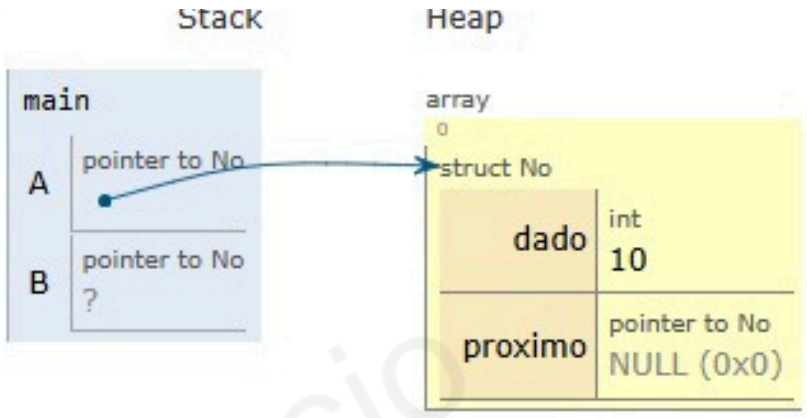


Lucas Palmaccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
```



Note: ? refers to an uninitialized value

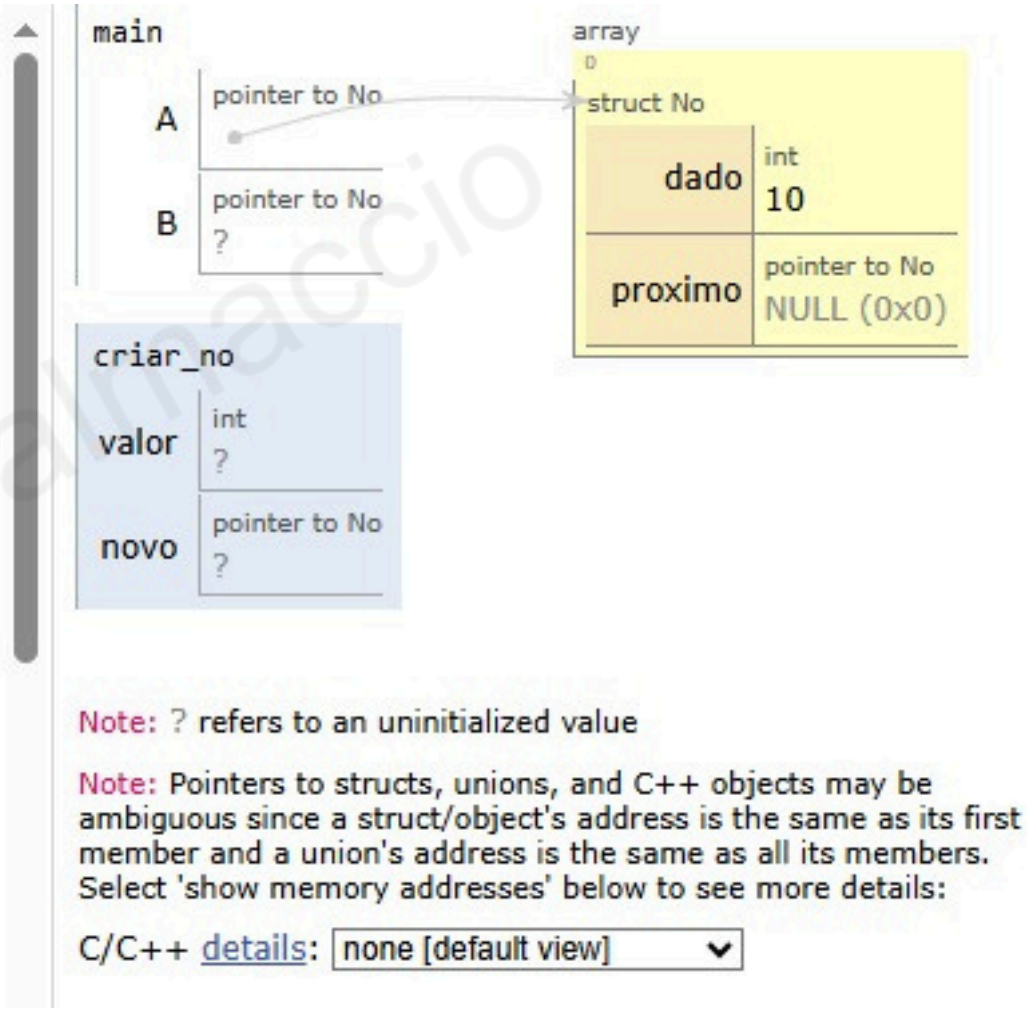
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

Lucas Palmeccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Note: ? refers to an uninitialized value

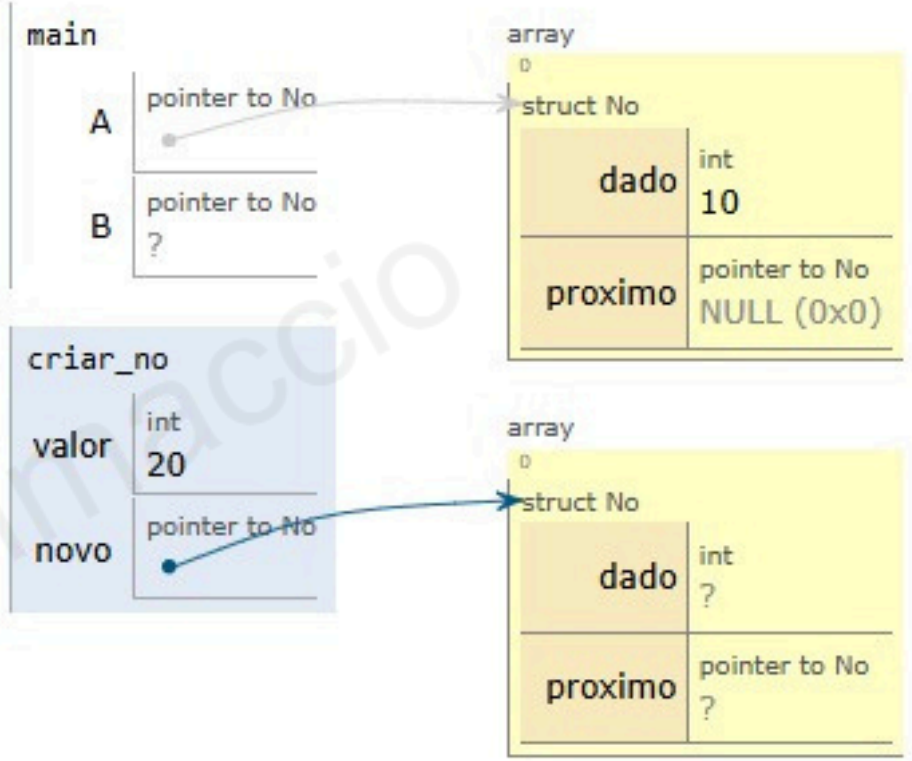
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

Lucas Palmaccio

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



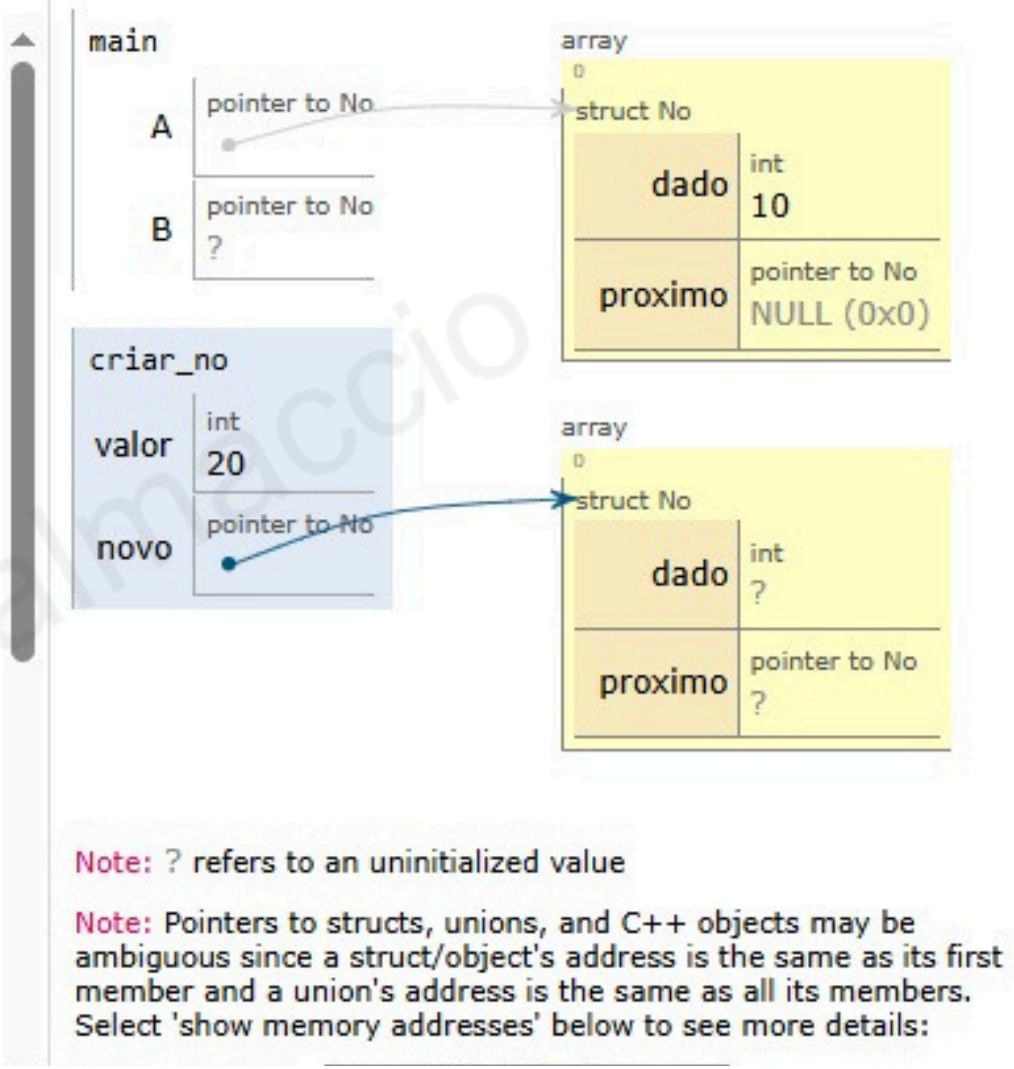
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

A variável ponteiro A reside na Stack e passa a armazenar o endereço do bloco de memória criado dinamicamente no Heap.

Lucas Palmaccio

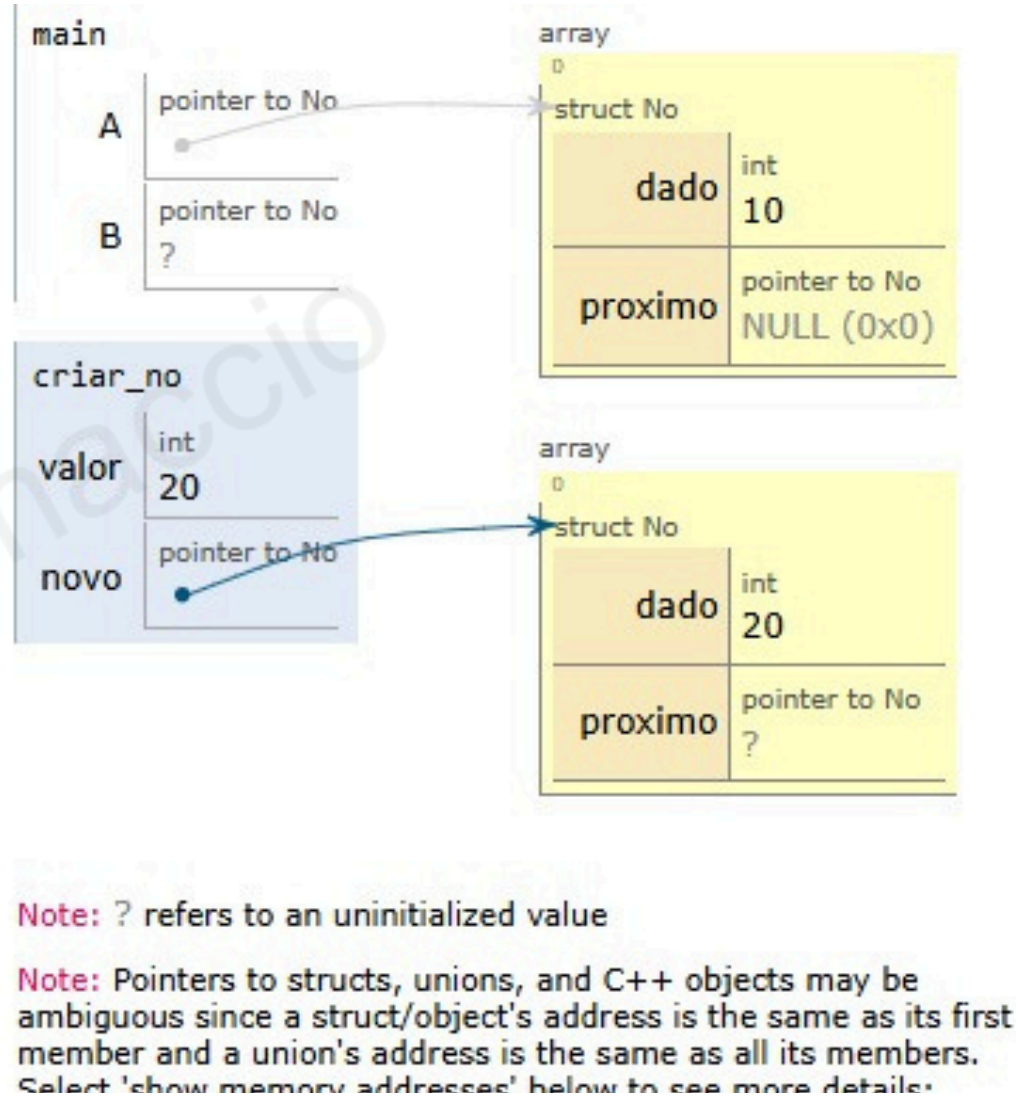
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



Lucas Palmaccio

Neste momento, A e B existem simultaneamente na memória, mas são estruturas isoladas. O ponteiro proximo de ambos aponta para NULL.

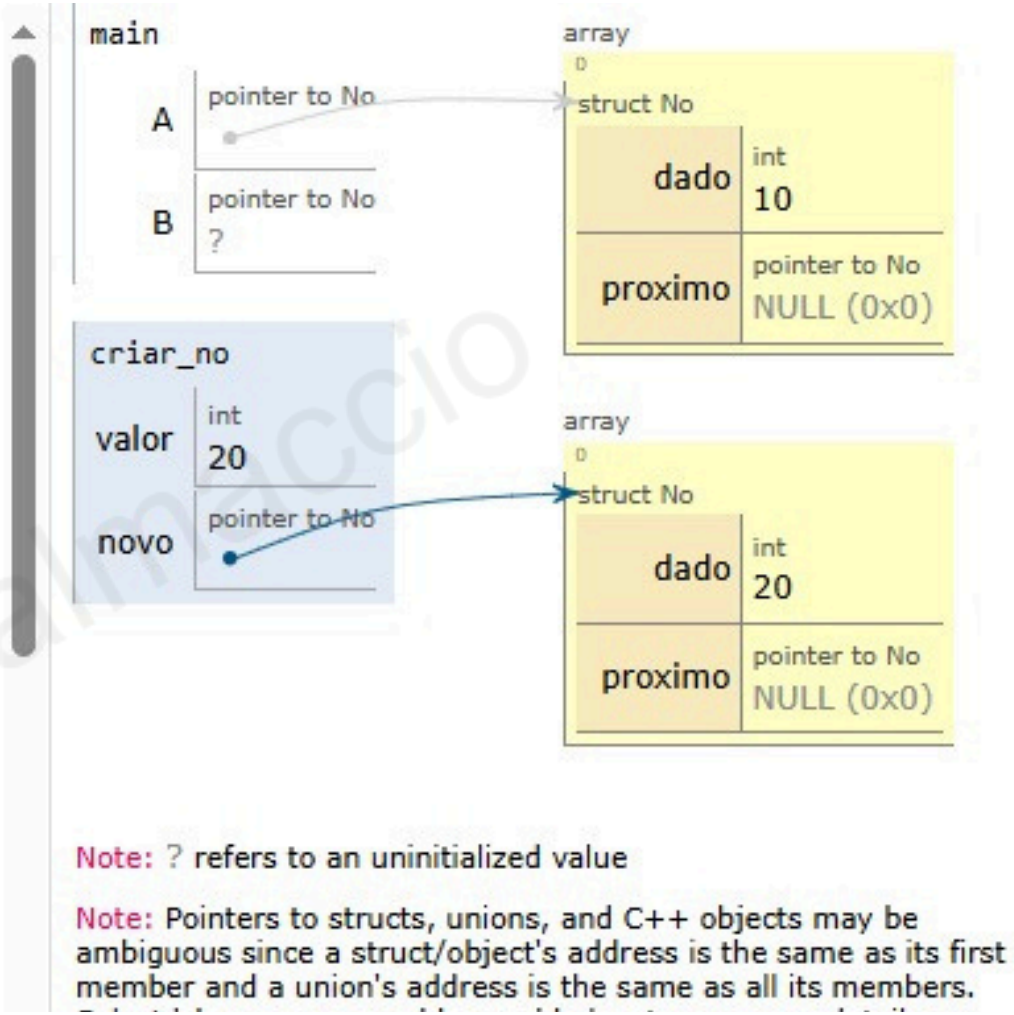
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

```



Lucas Palmaccio

Neste momento, A e B existem simultaneamente na memória, mas são estruturas isoladas. O ponteiro proximo de ambos aponta para NULL.

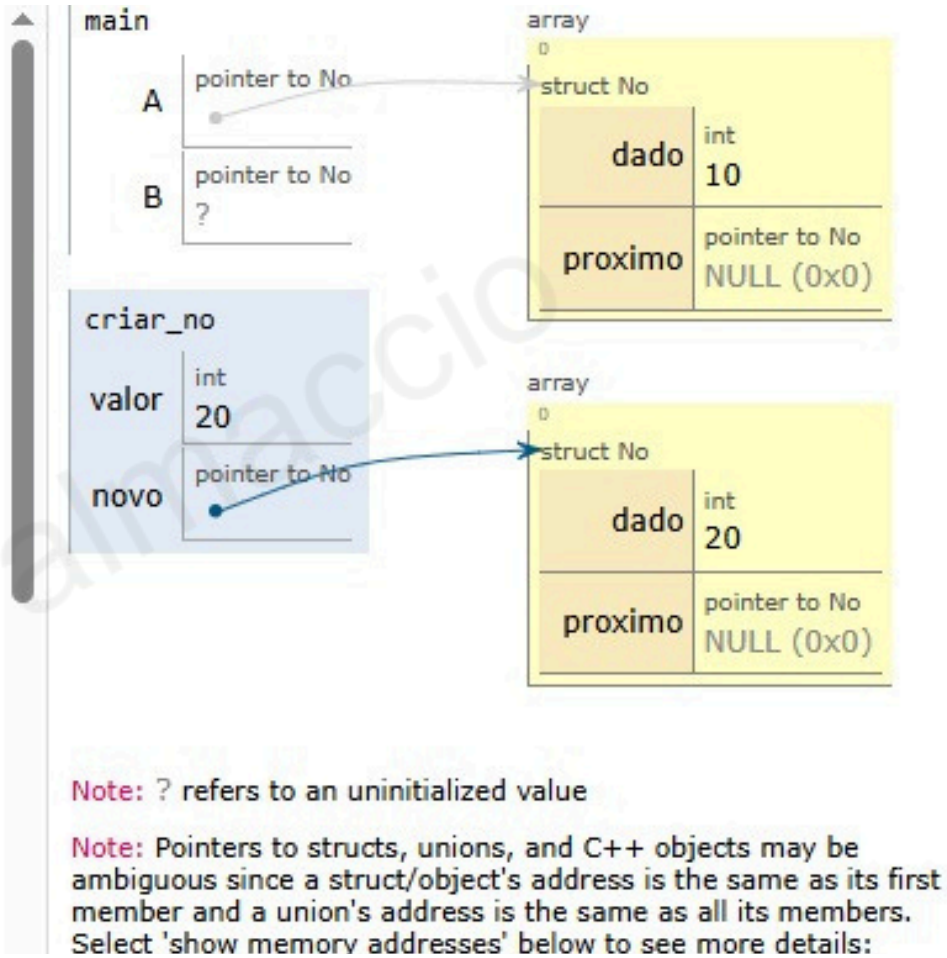
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



Lucas Palmaccio

Neste momento, A e B existem simultaneamente na memória, mas são estruturas isoladas. O ponteiro proximo de ambos aponta para NULL.

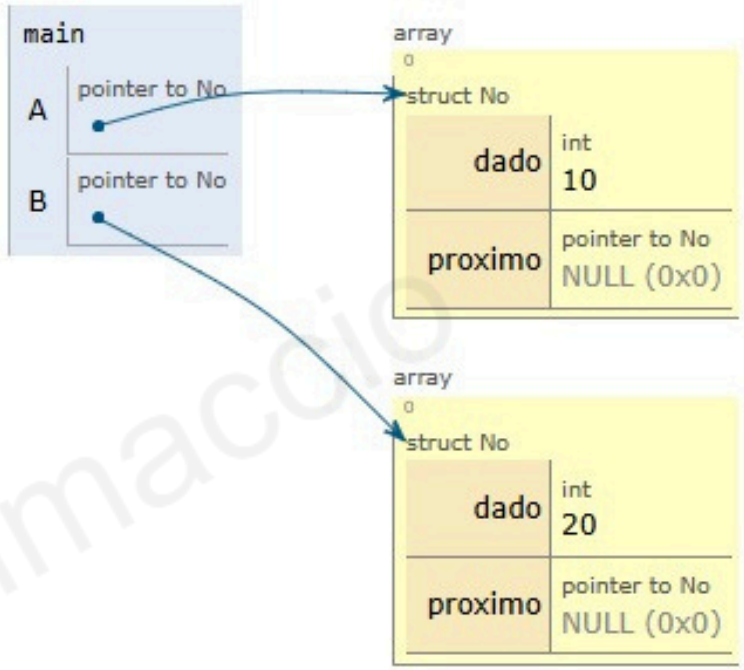
```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}
```



Lucas Palmaccio

Neste momento, A e B existem simultaneamente na memória, mas são estruturas isoladas. O ponteiro proximo de ambos aponta para NULL.

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```



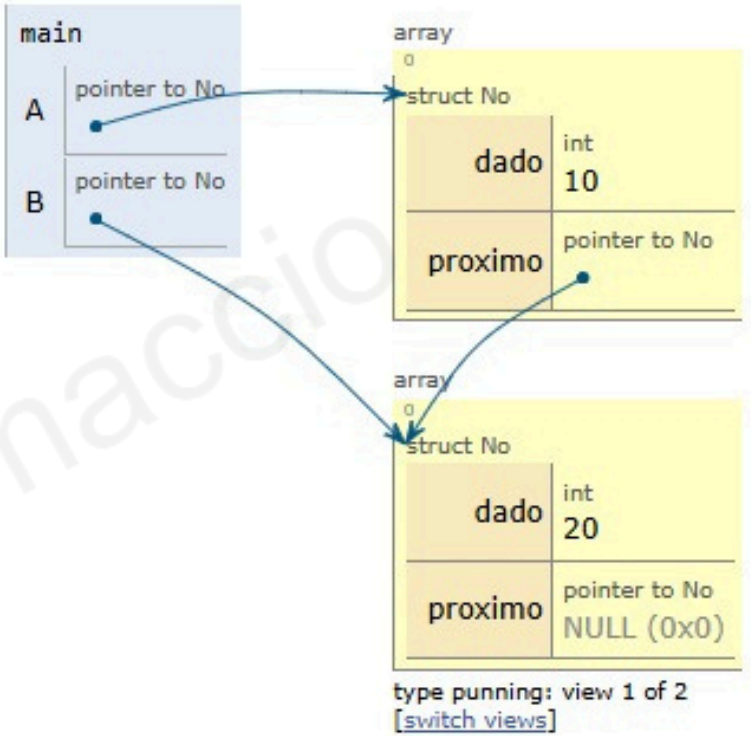
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

Lucas Palmaccio

O atributo proximo do nó A deixa de ser NULL e passa a guardar o endereço de memória do nó B, estabelecendo a ligação típica de uma Lista Encadeada.

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```

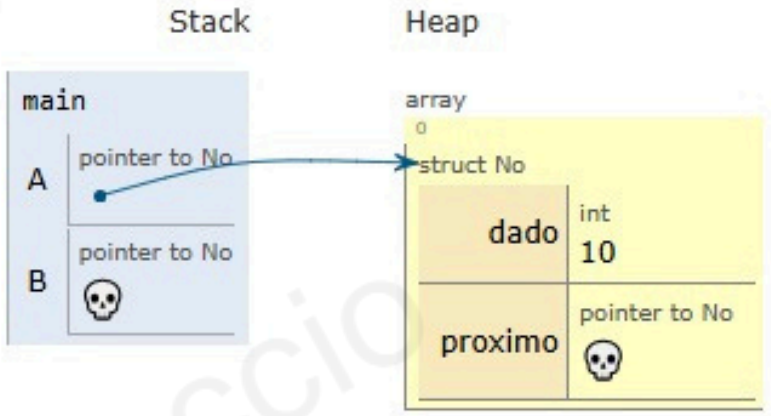


Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

O atributo proximo do nó A deixa de ser NULL e passa a guardar o endereço de memória do nó B, estabelecendo a ligação típica de uma Lista Encadeada.

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```



Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

Note: 💀 means a pointer points to memory that is either unallocated or misaligned with data boundaries. 💀 locations are *approximate* and may not match the pointer's real address. Select 'byte-level view of data' below to see more details:

C/C++ details: none [default view]

Lucas Palmaccio

A boa prática para evitar falhas de segmentação é atribuir A->proximo = NULL; imediatamente após o free(B)

Conclusão Técnica: O Problema do Dangling Pointer

A situação em que o ponteiro A perde a referência válida após o comando free(B) é tecnicamente conhecida como a criação de um Dangling Pointer (Ponteiro Pendente). Quando a memória de B é devolvida ao sistema (Heap), A->proximo continua armazenando aquele endereço antigo e agora inválido.

Qualquer tentativa futura do programa de ler ou escrever em A->proximo resultará em um comportamento indefinido, frequentemente causando o travamento do sistema (Segmentation Fault).

Boa Prática de Resolução: Para garantir a integridade da estrutura de dados e evitar acessos indevidos à memória, deve-se sempre neutralizar o ponteiro imediatamente após a liberação, adicionando a instrução: A->proximo = NULL;.

Lucas Palmarcio